

Activity 3: AI-Assisted TDD for C++

Course: AI Vibe Coding for iOS Mobile Apps Development (TGS-2023037545)

Mapping: K2, A2

Estimated time: 75 minutes

Objective

Use an evidence-bound AI prompt to implement a rule through Red-Green-Refactor.

Deliverable

Prompt contract, failing trace, minimal patch and passing trace

Tools

AI coding assistant, CTest, diff review

Included files

- `starter/failing-test.cpp`
- `starter/prompt-contract.txt`
- `solution/prompt-contract.txt`
- `solution/review-checklist.txt`

Before you begin

- Work only with the supplied mock data. Do not add credentials, personal data or signing files.
- Keep the portable C++ core free of Swift, Objective-C and Apple framework dependencies.
- Record the initial state before editing so that verification shows a meaningful change.
- If using an AI coding assistant, treat its output as a proposal and inspect the resulting diff.

Procedure

1. Run `cd activities/03-ai-assisted-tdd` and inspect `starter/failing-test.cpp`, `starter/prompt-contract.txt` and the completed examples under `solution/`.
2. Create `workspace/`, then copy the header and implementation produced in Activity 2 into it. If that output is unavailable, copy the corresponding files from `../02-portable-cpp-domain-core/solution/include` and `../02-portable-cpp-domain-core/solution/src`.
3. Copy `starter/failing-test.cpp` into `workspace/` and compile it with `c++ -std=c++20 -Wall -Wextra -Werror -I workspace workspace/BudgetPolicy.cpp workspace/failing-test.cpp -o workspace/red-test`.
4. Run `workspace/red-test`; expect a non-zero exit or assertion failure if the overspend rule is missing. Save the terminal output and exit status in `workspace/red-trace.txt` before changing production code.
5. Copy `starter/prompt-contract.txt` to `workspace/prompt-contract.txt`. Complete Outcome, Relevant context, Protected interfaces, Constraints, Exact verification commands and Finish condition.

6. Give the completed contract plus only the relevant header, implementation and failing test to the AI coding assistant. Instruct it to propose the smallest patch and not to change the method signature or cents unit.
7. Review the proposal before applying it. Reject changes that introduce floating point, alter unrelated formatting, remove the failing assertion or add an Apple dependency.
8. Apply the accepted lines to ``workspace/BudgetPolicy.cpp``. Capture a unified review artifact with ``diff -u ../02-portable-cpp-domain-core/solution/src/BudgetPolicy.cpp workspace/BudgetPolicy.cpp > workspace/review.diff || true``.
9. Recompile using the same command, run ``workspace/red-test``, and record the now-zero exit status in ``workspace/green-trace.txt``.
10. Run the complete Activity 2 regression suite against the changed implementation, not only the new overspend test. Expect valid, zero, negative and over-budget cases to pass.
11. Complete a copy of ``solution/review-checklist.txt``. Every unchecked item requires a repair followed by focused and regression reruns.
12. Retain prompt contract, red trace, reviewed diff, green trace and checklist. These five artifacts demonstrate Red-Green-Refactor rather than an unverified AI-generated result.

Verification

Evidence includes a meaningful initial failure, a scoped diff and a fully passing suite.

Capture the command output, test result, matrix or screenshot named by the deliverable. A statement such as “it works” is not sufficient evidence.

Troubleshooting

If the test passed before the change, strengthen the assertion before asking for implementation.

Check the earliest failing layer first: portable C++ domain test, then Objective-C++ contract, then Swift state and finally the visible interface.

Cleanup

Keep prompt and evidence; revert experimental changes outside the accepted patch.

Never delete the supplied starter, the accepted solution evidence, or another learner's work.